CLASE 19

Métodos y Encapsulación

"Proteger tus datos como un banco protege tu dinero"

- ¿Por qué proteger los datos de un objeto?
- Atributos "privados" en Python
- @property - acceso controlado
- Crear clases más seguras y profesionales

Repaso - Lo que ya sabemos

CLASE	El plano (define la estructura)
OBJETO	La cosa real (construida del plano)
__init__	Constructor (se ejecuta al crear)
self	"Este objeto" (referencia a sí mismo)
Atributos	Los DATOS del objeto
Métodos	Las ACCIONES del objeto

Ejemplo mental

Clase: CuentaBancaria

Atributos:
titular, saldo

Métodos:
depositar()
retirar()

El problema: Datos desprotegidos

Situación actual:

LEER los atributos → OK, sin problema

MODIFICAR los atributos → ¡PELIGRO!

Ejemplo del mundo real:

Tu cuenta de banco tiene $1,000

¿Qué pasa si alguien puede hacer
tu_saldo = 1,000,000 directamente?

Sin verificación, sin registro, sin permiso...

DESASTRE

Lo que necesitamos:

- Controlar QUIÉN puede modificar los datos
- Validar CÓMO se modifican
- Registrar CUÁNDO se modifican

Encapsulación

"Esconder los datos internos y controlar cómo se acceden"

Analogía: El cajero automático

NO puedes:

- Meter la mano en la caja fuerte
- Cambiar tu saldo directamente
- Modificar los registros del banco

SÍ puedes:

- Usar los botones del cajero
- El cajero VALIDA tu PIN
- El cajero VERIFICA fondos
- El cajero REGISTRA la transacción

Datos escondidos + Acceso controlado = ENCAPSULACIÓN

Convención de privacidad en Python

Python confía en los programadores (somos adultos responsables)

`atributo`

Público *Luz verde*

Acceso libre, sin restricciones

`_atributo`

"Privado" *Precaución*

No tocar desde afuera
(pero técnicamente SÍ se puede)

`__atributo`

"Muy privado" *Alto total*

Python lo renombra
(más difícil de acceder)

El guión bajo es una SEÑAL para otros programadores: "no toques esto"

Ejemplo: Proteger el saldo

ANTES (peligroso)

```
self.saldo = saldo
```

```
cuenta.saldo = 1000000  ✓
```

(cualquiera puede cambiar)

→

DESPUÉS (protegido)

```
self._saldo = saldo
```

```
cuenta._saldo = 1000000
```

(la señal dice "NO TOCAR")

El guión bajo es como un cartel de "Solo empleados". No hay candado, pero el mensaje es claro.

Pero... ¿cómo accedemos al saldo entonces?

→ Con métodos controlados (getters y setters)
→ **O mejor aún: con @property**

@property - Lo mejor de ambos mundos

El dilema:

`cuenta._saldo`	→ Feo y rompe la convención
`cuenta.consultar_saldo()`	→ Funciona pero es largo
`cuenta.saldo`	→ ¡Esto queremos! Pero protegido

@property nos permite:

`Escribir: cuenta.saldo`

Pero que internamente ejecute un MÉTODO

que puede validar, calcular, proteger...

Parece un atributo → Se comporta como un método

@property completo: getter y setter

GETTER (leer)

```
@property
def saldo(self):
    return self._saldo
```

Permite: valor = cuenta.saldo

SETTER (escribir)

```
@saldo.setter
def saldo(self, valor):
    # Aquí puedes VALIDAR
```

Permite: cuenta.saldo = 5000

El setter es OPCIONAL:

Sin setter → Solo lectura (más seguro)
Con setter → Lectura y escritura (con validación)

El setter es tu guardia de seguridad. Revisa todo antes de dejar pasar.

Ejemplo: Persona con edad protegida

Validaciones que podemos hacer:

- La edad no puede ser negativa
- La edad no puede ser mayor a 150
- El nombre no puede estar vacío
- El email debe tener @

Beneficios:

- ✓ Datos siempre válidos
- ✓ Errores detectados ya
- ✓ Código más confiable
- ✓ Menos bugs

Validar en el setter = Prevenir problemas ANTES de que ocurran

¿Cuándo usar @property?

USAR @property cuando:

- ✓ Necesitas validar datos al asignar
- ✓ Quieres calcular un valor (no almacenarlo)
- ✓ Quieres controlar lectura/escritura
- ✓ Podrías necesitar agregar lógica después

NO necesitas @property cuando:

- ✗ El atributo no necesita validación
- ✗ Es un dato simple sin restricciones
- ✗ No hay lógica especial

Ejemplo de propiedad calculada:

```
@property
def nombre_completo(self):
    return f"{self.nombre} {self.apellido}"
```

No se almacena, se CALCULA cada vez que se pide

EJERCICIO 1: Clase Producto

Crear clase Producto con:

Atributos:

- nombre (público)
- _precio (privado con @property)
- _stock (privado con @property)

Validaciones precio:

- No puede ser negativo
- No puede ser mayor a 10,000

Validaciones stock:

- No puede ser negativo
- Debe ser número entero

Propiedad calculada:

- valor_inventario = precio x stock

Pruebas esperadas:

```
Producto("Laptop", 999, 10)
```

→ Funciona

```
producto.precio = -50
```

→ Error

```
producto.stock = 5.5
```

→ Error (no entero)

```
producto.valor_inventario
```

→ 9990

Equipos de 2-3 | Tiempo: 12 minutos

BREAK

10 MINUTOS

Volvemos con la práctica final:
CuentaBancaria profesional

Proyecto: CuentaBancaria Profesional

Atributos protegidos

`_titular`
(solo lectura después de crear)

`_saldo`
(solo via depositar/retirar)

`_historial`
(registro de transacciones)

Métodos

`depositar(cantidad)`
con validación

`retirar(cantidad)`
con validación y fondos

`transferir(otra, cant)`
mover entre cuentas

Properties

`titular`
(solo getter)

`saldo`
(solo getter)

`historial`
(solo getter, copia)

El saldo NO tiene setter. Solo se modifica a través de depositar() y retirar()

Construyendo paso a paso

1. **Estructura básica**
   __init__ con atributos privados, historial vacío
2. **Properties solo lectura**
   titular, saldo, historial (solo getter)
3. **Método depositar**
   Validar positiva, actualizar saldo, registrar
4. **Método retirar**
   Validar positiva, verificar fondos, registrar
5. **Método transferir**
   Retirar de una, depositar en otra (bonus)

Vamos a VS Code

Probando la seguridad

X	`cuenta.saldo = 1000000`	→ AttributeError: can't set attribute
X	`cuenta.titular = "Hacker"`	→ AttributeError: can't set attribute
X	`cuenta.retirar(-100)`	→ ValueError: cantidad debe ser positiva
X	`cuenta.retirar(999999)`	→ ValueError: Fondos insuficientes
X	`cuenta.historial.append("Hack")`	→ No afecta el historial real (es una copia)

La cuenta está PROTEGIDA - Esto es encapsulación en acción

EJERCICIO 2: Clase Empleado

Atributos protegidos:

_nombre
lectura/escritura, no puede estar vacío

_salario
lectura/escritura, mínimo $500, máximo $50,000

_departamento
solo lectura después de crear

Métodos:

```
aumentar_salario(porcentaje)
info() - muestra datos del empleado
```

Propiedad calculada:

```
salario_mensual = salario / 12
```

Pruebas esperadas:

```
emp.nombre = ""
```
→ Error (vacío)

```
emp.salario = 300
```
→ Error (menor al mínimo)

```
emp.aumentar_salario(10)
```
→ Aumenta 10%

```
emp.departamento = "Otro"
```
→ Error (solo lectura)

Equipos de 2-3 | Tiempo: 15 minutos

Revisemos la solución

Constantes para límites

```
SALARIO_MINIMO = 500
SALARIO_MAXIMO = 50000
```

Departamento sin setter

```
= solo lectura automáticamente
```

aumentar_salario usa el setter

```
self.salario = nuevo_salario
(para que valide el máximo)
```

Propiedad calculada

```
salario_mensual = salario / 12
```

Formato de números

```
${valor:,.2f} → $30,000.00
```

Resumen - Métodos y Encapsulación

Concepto	Descripción
`Encapsulación`	Proteger datos, controlar acceso
`_atributo`	Convención: "privado, no tocar"
`@property`	Getter - método disfrazado de atributo
`@x.setter`	Setter - validar al asignar

Patrón común:

```
self._dato
@property getter
@setter (opcional)
métodos para operaciones
```

Cuándo usar qué:

Solo lectura

`@property sin setter`

Lectura/escritura

`@property + @setter`

Solo métodos

`depositar(), retirar()`

Próxima clase: Herencia → "Crear clases basadas en otras clases"